

# **LABORATORIO DI SISTEMI OPERATIVI**

## ***Shared memory***

fabio.rossi@unipg.it

075 5855005

# IPC (interprocess communication)

- vari strumenti per la comunicazione tra processi:
  - signal
  - socket
  - pipe
  - **shared memory**
  - etc.

# Shared Memory

- Due implementazioni: *POSIX* e *SYSTEM V*
- *POSIX*: interfaccia più semplice e meglio congegnata; ma meno diffusa, specialmente in sistemi vecchi (*man 7 shm\_overview*)
- *SYSTEM V*: *API* più datata ma presente anche in sistemi molto vecchi (*POSIX* potrebbe non esserci) (*man 7 sysvipc*)
- **tratteremo solo *POSIX***

# Shared Memory (*POSIX*)

- processi che condividono un'area di memoria e, conseguentemente, attraverso essa possono comunicare/sincronizzarsi
- tipicamente i processi devono sincronizzare l'accesso alla memoria condivisa, per esempio con dei semafori
- linkare con **-lrt** (*librt*) se necessario
- hanno persistenza nel *kernel*, le aree non deallocate permangono sino al prossimo *shutdown*

# Shared Memory in Linux

- in *Linux* sono visibili nel filesystem virtuale */dev/shm*
- possiamo impostare permessi

Shared Memory: interfacce dell'API *POSIX* (da *man 7 shm\_overview*)

- ***shm\_open(3)***: Create and open a new object, or open an existing object. This is analogous to *open(2)*. The call returns a file descriptor for use by the other interfaces listed below.
- ***ftruncate(2)***: Set the size of the shared memory object. (A newly created shared memory object has a length of zero.)
- ***mmap(2)***: Map the shared memory object into the virtual address space of the calling process.
- ***munmap(2)***: Unmap the shared memory object from the virtual address space of the calling process.

## Shared Memory: interfacce dell'API *POSIX* (da *man 7 shm\_overview*)

- ***fstat(2)***: Obtain a stat structure that describes the shared memory object. Among the information returned by this call are the object's size (*st\_size*), permissions (*st\_mode*), owner (*st\_uid*), and group (*st\_gid*).
- ***close(2)***: Close the file descriptor allocated by *shm\_open(3)* when it is no longer needed.
- ***shm\_unlink(3)***: Remove a shared memory object name.
- ***fchown(2)***: To change the ownership of a shared memory object.
- ***fchmod(2)***: To change the permissions of a shared memory object.

# Shared Memory

```
#include <sys/mman.h>
#include <sys/stat.h>          /* For mode constants */
#include <fcntl.h>              /* For O_* constants */

int shm_open(const char *name, int oflag, mode_t mode);
```

- analoga ad *open(2)*
- apre una *shared memory* (o la crea se non esiste e si specifica il flag **O\_CREAT**)
- torna un *file descriptor* (o -1 per errore settando *errno*)
- **name** identifica la *shared memory*. Per portabilità dovrebbe essere nella forma **"/somename"**
- **oflag** devono comprendere uno tra **O\_RDONLY** e **O\_RDWR**. Tra i possibili **oflag** vi sono **O\_CREAT** ed **O\_TRUNC**
- **mode** specifica i permessi, come in *open(2)*



# Shared Memory

```
#include <sys/mman.h>
#include <sys/stat.h>          /* For mode constants */
#include <fcntl.h>             /* For O_* constants */

int shm_open(const char *name, int oflag, mode_t mode);
```

- un'area di nuova creazione ha dimensione iniziale 0 byte
- la dimensione può essere variata con *ftruncate(2)*
- i *bytes* di una nuova area sono inizializzati a 0 (cod. *ASCII*)

# Shared Memory

```
#include <sys/mman.h>
#include <sys/stat.h>          /* For mode constants */
#include <fcntl.h>             /* For O_* constants */

int shm_unlink(const char *name);
```

- analoga ad *unlink(2)*
- rimuove un'area di memoria creata precedentemente con *shm\_open(2)*. O meglio....
- rimuove "il nome" dall'elenco degli oggetti "*shared memory*" e, quando tutti i processi hanno fatto *l'unmapping* dell'area, dealloca e distrugge il suo contenuto
- in altre parole, si può richiamare *shm\_unlink()* anche quando ancora vi siano processi che usano l'area

# Shared Memory

```
#include <unistd.h>
```

```
int ftruncate(int fd, off_t length);
```

- in generale tronca un file alla dimensione specificata da **length**
- nel nostro caso stabilisce la dimensione della *shared memory* (considerate che *shm\_open(2)* torna un *file descriptor*....)
- torna 0 per successo, -1 per errore (***errno*** settata)

# Shared Memory

```
#include <sys/mman.h>
```

```
void *mmap(void *addr, size_t length, int prot,  
           int flags, int fd, off_t offset);
```

- in generale mappa file o device nello spazio di indirizzamento virtuale del chiamante. Nel nostro caso mappa (anche) *shared memory*
- torna un puntatore all'area mappata
- **addr**: indirizzo iniziale nello spazio di indirizzamento per la mappatura. Se *NULL*, sceglie il *kernel* dove mappare (più portabile)
- **length**: lunghezza dell'area mappata
- **offset**: l'area mappata è inizializzata col contenuto della *shared memory*, referenziata da **fd**, a partire da **offset**\* per lunghezza **length**

(\*): *offset* si riferisce alla posizione nel/nella file/*shared memory* mappato/a. Deve essere un multiplo di "*page size*", che viene ritornato (cioè può essere ottenuto) da `sysconf(_SC_PAGESIZE)`

# Shared Memory

```
#include <sys/mman.h>
```

```
void *mmap(void *addr, size_t length, int prot,  
           int flags, int fd, off_t offset);
```

- **prot**: indica i permessi sull'area (non deve essere in conflitto con il modo di apertura della *shared memory*). Può essere:
  - PROT\_NONE: l'area non può essere acceduta

oppure una combinazione di:

- PROT\_EXEC: l'area ha il permesso di esecuzione
- PROT\_READ: l'area può essere letta
- PROT\_WRITE: l'area può essere scritta

# Shared Memory

```
#include <sys/mman.h>
```

```
void *mmap(void *addr, size_t length, int prot,  
           int flags, int fd, off_t offset);
```

- **flags**: determinano, tra l'altro, se le modifiche all'area mappata sono o meno visibili agli altri processi che la usano (e nel caso di file, se le modifiche sono trasferite al file sottostante):
  - **MAP\_SHARED**: le modifiche sono visibili agli altri processi
  - **MAP\_PRIVATE**: modifiche non visibili agli altri processi
  - **MAP\_SHARED\_VALIDATE**: è un'estensione di Linux (i due precedenti sono definiti da *POSIX*). Come **MAP\_SHARED** ma, mentre con **MAP\_SHARED** eventuali altri *flags* "sconosciuti" sono ignorati, con **MAP\_SHARED\_VALIDATE** il *kernel* controlla che gli altri *flags* specificati siano validi
  - in Linux ce ne sono vari altri... (sezione "*The flags argument*" di **man 2 mmap**)

# Shared Memory

```
#include <sys/mman.h>
```

```
void *mmap(void *addr, size_t length, int prot,  
           int flags, int fd, off_t offset);
```

- **fd**: il *file descriptor* tornato da *shm\_open(2)*
- dopo che *mmap(2)* è ritornata il *file descriptor* **fd** può essere chiuso senza invalidare il *mapping*

# Shared Memory

```
#include <sys/mman.h>
```

```
void *mmap(void *addr, size_t length, int prot,  
           int flags, int fd, off_t offset);
```

- ritorna un puntatore all'area mappata. In caso di errore torna **MAP\_FAILED** (cioè **(void \*) -1**) e setta *errno*
- il *mapping* è preservato dopo *fork(2)*



# Shared Memory

- in *man 3 shm\_open* trovate un esempio